

THAMMASAT UNIVERSITY

ScamGraph AI

Evidence-Linked and Explainable Scam-Risk Assessment for Thai-English Messages and URLs

Technical Research Manuscript

Kitsanapuch Chuaibamrung

Thanakorn Morasilp

Phutawan Sathusarn

Achiraya Sungsriin

Software Engineering

Bachelor's Degree Program

12 October 2026

Abstract

Scam messages combine persuasive language, suspicious links, payment identifiers, and impersonation cues. A useful screening system must explain its assessment without presenting incomplete evidence as a confirmed allegation. This paper describes ScamGraph AI, a Thai-English web and mobile prototype that combines character-level text classification, lexical URL classification, provenance-aware evidence lookup, and explicit risk policies. Backend OCR and QR decoding extend the input workflow; graph neighborhoods, anomaly scores, and semantic retrieval provide context rather than a learned fraud verdict. We examine saved model artifacts trained on 400 balanced synthetic records, organized into 150 connected groups and partitioned into 230 training, 80 validation, and 90 frozen test records. At validation-selected thresholds, text and URL classifiers achieved test accuracies of 94.44% and 96.67%, and F1 scores of 0.9425 and 0.9663, respectively. These are component-level results on demonstration data, not estimates of real-world scam-detection accuracy. We explain all reported metrics, the grouping and threshold-selection protocol, and the recorded ML, API, security, client, PWA, and iOS verification methods. We distinguish reproduced model evidence from mocked integrations and unverified physical-device capabilities. External validation, calibration, representative human studies, and deployment-scale testing remain necessary before public reliability claims.

Keywords: scam-risk assessment; Thai-English text; explainable machine learning; URL analysis; data leakage; cross-platform prototype.

1. Introduction

Scam screening is a decision-support problem with consequences beyond binary classification. A suspicious message may be legitimate in context, and an unfamiliar phone number may have no connected history. An application should therefore communicate the evidence it has, the evidence it lacks, and the difference between a model response and verified incident information. ScamGraph AI targets this interaction through a shared web/mobile interface and a structured assessment API.

The contribution is a documented integration and evaluation of existing learning methods, rather than a new classification algorithm: (i) multiple input workflows share an auditable result contract; (ii) model contributions and source relationships retain their units and provenance; and (iii) experimental metrics are separated from application risk labels and engineering test outcomes. The system is a hackathon prototype with synthetic training data. No claim of production readiness, calibrated fraud probability, or superiority over deployed scam-detection services is made.

2. Related Work and Positioning

TF-IDF weights combine term occurrence with collection-level rarity [1]. Character n-grams are useful for a prototype spanning Thai and English because they do not require a language-specific word-segmentation pipeline, although this choice alone does not establish multilingual robustness. The implementation uses scikit-learn transformations and logistic regression [2]. Its URL classifier uses XGBoost [3] over structured lexical features, while Isolation Forest [4] supplies a separate description of unusual URL patterns.

SHAP provides an additive framework for explaining individual predictions [5]. Sentence-BERT [6] and multilingual knowledge distillation [7] motivate sentence representations for retrieval; the optional encoder is the officially documented 384-dimensional multilingual MiniLM model [8]. These references support the methods used, not the project's detection performance. The pretrained model was not fine-tuned on verified scam incidents in this study.

Dataset dependence and leakage can inflate apparent performance [9]. We therefore examine campaign, domain, and normalized-text grouping and explicitly retain the limitations of synthetic templates. Text and

URL components use different input representations; their scores below are not a controlled comparison establishing that one learning algorithm is superior. No external benchmark, baseline ablation, or graph neural network experiment is included.

3. System Architecture

Expo/React Native and TypeScript implement shared web and native screens. FastAPI validates requests through Pydantic, coordinates model inference and evidence lookup, and returns an OpenAPI-described contract. SQLAlchemy persists owned history, reports, evidence, and jobs. SQLite is used for the recorded local demonstrations; Docker Compose specifies a PostgreSQL deployment path. A static Vercel frontend configuration requires a separately hosted HTTPS API [10].

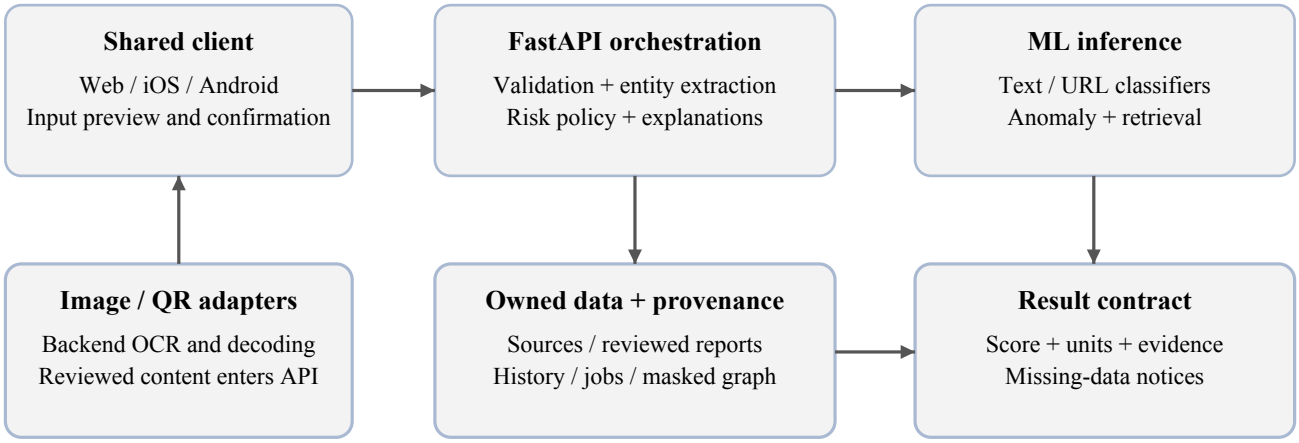


Figure 1. Prototype architecture. OCR/QR adapters normalize inputs before explicit review. Graph and retrieval context are distinct from the classifier scores. A shared result contract serves each client.

The response exposes risk score and level, extracted entities, evidence-backed reasons, model version, sample status, contributions, anomaly context, graph provenance, missing data, and suggested next steps. Guests can assess content. Explicit saving, reporting, evidence upload, and batch analysis use authenticated ownership checks. Scores and graph links are presented as screening support, not proof of wrongdoing.

4. Machine-Learning Components

4.1 Text representation and classifier

The text pipeline removes HTTP(S) URLs before inference and transforms remaining content into character 2-5-gram TF-IDF vectors. The vectorizer is fitted only on training rows, with minimum document frequency 2, a maximum vocabulary cap of 30,000, sublinear term frequency, and no accent stripping. The cap is not a measurement of the fitted vocabulary size. Logistic regression uses balanced class weights, C = 3, and up to 2,000 iterations. The stored training library version is scikit-learn 1.6.1; its default solver and penalty are lbfgs and L2 [10].

For feature vector x, the model margin and output are:

$$m_{text} = b + \sum_j w_j x_j, \quad p_{text} = 1 / (1 + \exp(-m_{text})). \quad (1)$$

An individual text contribution is $w_j x_j$. The intercept plus all contributions reconstructs the raw margin. The interface shows up to eight contributions ranked by absolute magnitude. This is an exact linear decomposition, not SHAP. Contributions use log-odds units; they are neither percentage-point changes in the application score nor causal evidence of a scam.

4.2 URL representation and classifier

URLs are parsed locally without fetching their destination. Twenty lexical features include URL, host, path and query lengths; punctuation, digits and path depth; subdomains; IP-host and punycode indicators; HTTPS; suspicious keywords; host and URL entropy; user-info/encoding/port indicators; and brand token/spelling similarity using difflib.SequenceMatcher. This is not Levenshtein distance or verified brand ownership. Brand and keyword lists are finite heuristics, not live reputation, domain-age, ownership, or carrier services. HTTPS is an input feature and is not treated as a safety guarantee.

Component	Stored/configured settings
XGBoost	120 estimators; depth 3; learning rate 0.06; min_child_weight 2; reg_lambda 2; row/column subsampling 0.9; histogram tree method; logloss metric; seed 2026; two workers.
Isolation Forest	160 estimators; contamination 0.12; seed 2026; two workers; fitted on 116 normal training URLs only.
Similarity retrieval	Up to three positive training examples; optional local 384-dimensional multilingual MiniLM encoder; explicit lexical TF-IDF cosine fallback.

Table 1. Configured learning and context components. Hyperparameters describe this prototype, not optimized settings for real incident traffic.

Tree SHAP explains XGBoost in raw-margin/log-odds space. The full base value plus attribution sum is checked against the model margin with relative and absolute tolerances of 10^{-4}; up to eight contributions are displayed. For multiple URLs, the maximum available URL output is used. A truncated explanation is accompanied by the full-sum metadata, so the displayed terms are not assumed to reconstruct the entire margin.

4.3 Context components

Isolation Forest describes departure from normal training URL patterns; no anomaly-to-fraud calibration is claimed. Sentence retrieval uses cosine similarity and a training-only positive reference collection. Its optional local model is pinned to a revision [8,10]. Similarity is contextual, not verification that the new input is fraudulent. Neither component has a saved standalone accuracy benchmark in this study.

5. Risk Policy, Graph Evidence, and Input Workflows

5.1 Explicit composition and abstention

The prototype does not train an ensemble over graph, history, anomaly, and similarity features. Let A be the available text/URL classifier outputs for an input. Its model screening score is:

$$S_{ML} = 100 \max(A), \text{ when A is nonempty.} \quad (2)$$

When A is empty, a zero composition baseline is used only to apply a qualifying evidence floor; it is not a "safe" model prediction. The no-evidence case abstains as described below.

Non-sample evidence can impose an explicit policy floor F, rather than another learned probability. Source-confirmed, moderator-reviewed community, and reported-source evidence use floors 85, 65, and 50, respectively. Sample history is excluded from these floors. The displayed score is:

$$S = \max(S_{ML}, F), \text{ with applicable components and policies disclosed.} \quad (3)$$

If no classifier or qualifying identifier evidence is available, the application returns INSUFFICIENT DATA. In particular, lack of history for a phone, account, or wallet does not become a "safe" verdict. Operational levels are LOW below 40, MEDIUM from 40 to below 70, and HIGH at 70 or above. These

levels are distinct from the component classification thresholds used to compute test metrics. The numerical score is explicitly uncalibrated.

5.2 Evidence graph semantics

A NetworkX MultiGraph links extracted entities, source records, and moderator-reviewed reports. Relationships carry evidence provenance, retrieval time, relation type, and sample status. Pending and rejected reports are excluded from the admitted threat graph. Two-hop neighborhoods and descriptive connected-component statistics support inspection. Input co-occurrence is separately labelled and excluded from risk features. A connection does not establish common ownership, criminal intent, or a verified fraud event [10].

5.3 OCR, QR and batch pathways

Input pathway	Processing and decision boundary
Images	Backend Tesseract Thai/English OCR, with optional macOS backend Vision fallback. Extracted text is previewed and editable before analysis; unreadable images receive an explicit failure state.
QR codes	OpenCV decoding; supported payment payloads use EMV TLV/CRC and PromptPay receiver parsing. The payload/receiver is reviewed before assessment. No link opening or payment is automatic.
CSV batches	Authenticated owned jobs validate 1-200 rows and expose progress and per-row results. Local threads/background tasks are a prototype mechanism, not a distributed queue.
Identifiers	Rule-based normalization for phone, bank/PromptPay, wallet and LINE identifiers. Evidence lookup may abstain; no external account-holder or telecom registry is connected.

Table 2. Input adapters and explicit review/abstention boundaries.

These adapters make the application a multi-input system. They do not constitute a trained multimodal image/text fusion model. OCR extraction quality, QR decoding success, and classification performance are separate outcomes and require separate evaluation datasets.

6. Dataset and Experimental Protocol

6.1 Provenance and composition

The saved artifacts use version *sample-v1-6c6c58eff5*, trained from 400 explicitly labelled synthetic demonstration rows: 200 normal and 200 scam. A read-only audit of the saved dataset found 200 Thai and 200 English rows, with 100 rows of each class per language. There are 200 authored campaigns, 320 distinct normalized text strings, and 400 distinct URLs, all hosted under reserved .test domains. These are generated examples, not victim reports, bank records, or adjudicated incidents. The metadata reports zero removed duplicate rows; this does not mean every text string is unique.

Split	Groups	Rows	Normal / scam	Thai / English
Training	90	230	116 / 114	112 / 118
Validation	30	80	40 / 40	42 / 38
Frozen test	30	90	44 / 46	46 / 44
Total	150	400	200 / 200	200 / 200

Table 3. Saved group and row allocation. Group proportions are 60/20/20%; row proportions are 57.5/20/22.5%, because connected groups have different sizes.

6.2 Dependence-aware partitioning

Records are connected transitively when they share a campaign identifier, conservative registrable-domain key, or casefolded whitespace-normalized exact text. Duplicate and conflicting-label validation occurs before partitioning. GroupShuffleSplit holds out 20% of groups for testing, then 25% of remaining groups for validation. The implementation searches seeds 2026-2125, rejecting single-class splits and minimizing differences in class prevalence across the partitions; it uses labels to balance allocation, not prediction performance.

The resulting frozen allocation was audited for coverage and disjointness. No pair of partitions shares a row identifier, campaign identifier, normalized exact text, or .test registrable domain; all 400 identifiers are covered. These checks reduce direct overlap. They do not remove semantic near-duplicates or shared generative templates. Conservative domain grouping is not a complete live public-suffix service [9,10].

6.3 Training, threshold selection and frozen evaluation

Vectorizer fitting, classifier fitting, normal-reference anomaly fitting, and retrieval reference construction use training rows only. No component output is stacked as another model's training feature. Classification thresholds are selected on validation scores by maximizing F1, breaking ties by precision and then distance to 0.5. Candidate thresholds include 0.5, validation score values, and 81 grid points from 0.1 to 0.9. The chosen thresholds are 0.71 for text and 0.50 for URLs.

Evaluation uses saved test identifiers and verifies the canonical dataset fingerprint before recomputing metrics. The stored fingerprint begins 6c6c58eff5; it binds the record content, not merely a filename. This paper reports saved artifacts and read-only consistency checks, not a new retraining experiment or a test-driven choice of an improved threshold.

7. Accuracy Measures and Quantitative Results

7.1 What each metric measures

For the positive class "scam," TP and FN count scam rows correctly and incorrectly classified, while TN and FP count normal rows correctly and incorrectly classified. Accuracy measures the fraction of all correctly classified rows; precision measures reliability among positive predictions; recall measures coverage of actual positive rows. F1 is their harmonic mean:

$$\text{Accuracy} = (TP + TN) / N; \quad \text{Precision} = TP / (TP + FP). \quad (4)$$

$$\text{Recall} = TP / (TP + FN); \quad F1 = 2PR / (P + R). \quad (5)$$

The artifact field named pr_auc is specifically scikit-learn average precision (AP), not trapezoidal integration of an interpolated curve. For precision P_k at recall R_k, ordered along increasing recall:

$$AP = \sum_k (R_k - R_{k-1}) P_k. \quad (6)$$

AP evaluates ranking over thresholds. It can be near 1 while a particular chosen threshold still misses positives. Accuracy percentages below are derived from the stored confusion matrices; other measures are saved artifact values. ROC-AUC is not recorded and is not substituted for AP.

Model / split	N	Threshold	Accuracy	Precision	Recall	F1	AP
Text / train	230	0.71	100.00%	1.0000	1.0000	1.0000	1.0000
Text / validation	80	0.71	98.75%	1.0000	0.9750	0.9873	0.9983
Text / test	90	0.71	94.44%	1.0000	0.8913	0.9425	0.9947
URL / train	230	0.50	100.00%	1.0000	1.0000	1.0000	1.0000
URL / validation	80	0.50	97.50%	0.9524	1.0000	0.9756	0.9926
URL / test	90	0.50	96.67%	1.0000	0.9348	0.9663	1.0000

Table 4. Component outcomes on the synthetic dataset. URL test AP rounds to 1.0000. Training results are in-sample diagnostics, not independent accuracy estimates.

7.2 Interpretation and limits of accuracy

On the frozen test set, the text model correctly classified 85/90 rows and the URL model 87/90. Both had zero false positives among 44 normal rows, but text missed 5/46 scam rows and URLs missed 3/46. Thus precision 1.0000 in this small sample does not imply zero false alarms in deployment. Perfect training scores and templated data make external validation especially important.

A 400-row balanced synthetic collection does not reproduce real scam prevalence, message diversity, evolving campaigns, URL obfuscation, dialects, or incidental legitimate uses of urgency/OTP language. Balanced test precision cannot be transferred unchanged to a lower-prevalence population. No statistically powered population study, uncertainty analysis, independent real-world test set, or calibration evaluation has been conducted. Reporting "96.67% accurate in detecting real scams" would be unsupported.

8. Error Analysis and Scope of the Reported Results

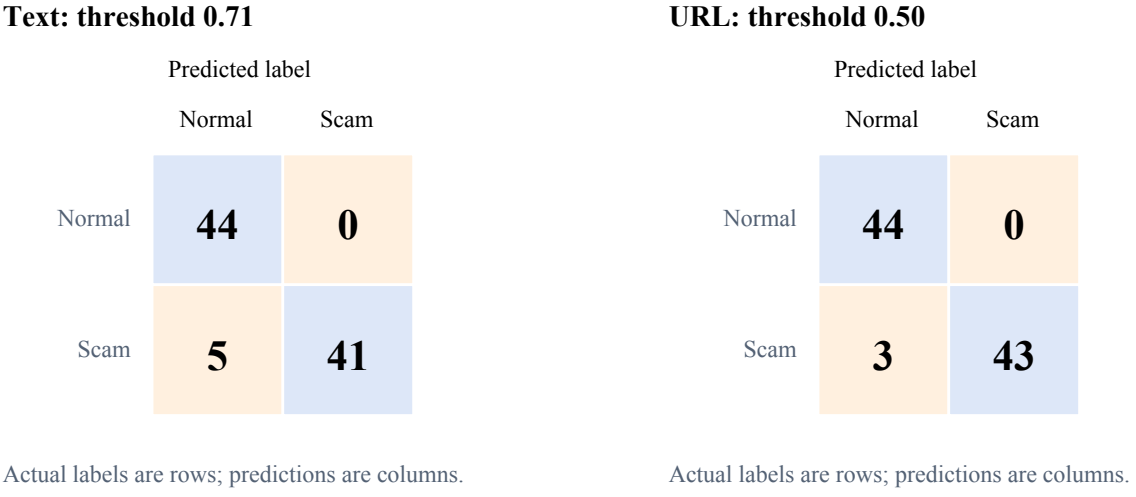


Figure 2. Frozen-test confusion matrices (normal/scam label order). Text: TN=44, FP=0, FN=5, TP=41. URL: TN=44, FP=0, FN=3, TP=43. These matrices describe only 90 synthetic records.

8.1 Operating-point behavior

At the text threshold, recall is 41/46 = 0.8913. At the URL threshold, recall is 43/46 = 0.9348. AP describes score ordering rather than this selected operating point; URL AP near 1 is compatible with three positive scores below 0.50. Changing thresholds after inspecting this test set would require a new independent evaluation set before claiming an improved generalization result.

Text and URL scores are evaluated on their own representations, so the difference in their F1 scores does not isolate an algorithm effect. The study has no controlled same-input baseline or modality ablation. No overall accuracy is saved for max-score fusion with evidence floors. There are also no measured accuracy outcomes for the graph, optional embeddings, OCR text extraction, QR decoding over a representative image corpus, LLM explanations, or caller identification on physical phones.

8.2 Example system behavior, not an additional benchmark

Recorded web and iOS smoke tests analyzed a synthetic urgent-OTP example and displayed the same screening score, 66.20, with explanations. The value is a reproducibility observation, not a probability of fraud or an aggregate accuracy metric. Application MEDIUM thresholds (40/70) and the classifier decision threshold (0.71) answer different questions; a 66.20 screening score must not be interpreted as 66.20% confidence or as an automatic contradiction of the classifier test label.

Evidence type	What this paper can support
Saved classifiers	Precision, recall, F1, AP and derived accuracy for the stored synthetic splits.
ML regression tests	Specified invariants, failures, margins, leakage checks and truthful fallback behavior.
API / client / native QA	Recorded functional behavior in the named test environment and fixtures.
Unmeasured field outcomes	No public deployment accuracy, live incident detection rate, human decision improvement, latency guarantee, or production-scale reliability estimate.

Table 5. Distinct evidence scopes for statistical and engineering claims.

Table 5 distinguishes software correctness from predictive effectiveness. Fallback and ownership tests verify specified behavior; they do not estimate detection sensitivity on new campaigns.

9. ML Verification Methods

The ML test source and artifact protocol [10] cover the following groups. They are regression and consistency checks, separate from the held-out classification experiment. The paper audits existing tests and QA records; it does not assert that every suite was newly rerun during manuscript preparation.

Test group	Inputs / procedure	Expected assertion
Data validation	Conflicting labels; exact duplicated content; shared campaigns and domains; transitive connections.	Reject inconsistent labels, normalize duplicates, and preserve connected groups.
Partition integrity	Generate sample partitions and compare group, campaign and domain membership.	No direct group leakage; required splits contain both classes. Saved-ID coverage is independently checked.
Lexical features	HTTPS, raw user-info, IP/punycode, host/path/encoding features.	HTTPS is not a safety rule; raw lexical cues survive normalization; URLs are not fetched.
Input sensitivity	Different normal/suspicious Thai and English inputs against fitted artifacts.	Scores change with content rather than hardcoded demonstration results.
Missing/partial models	Absent artifacts, empty input, URL-only input, or a failed URL model.	Return null/unavailable information and retain a valid available text component.
Attribution units	Compare XGBoost SHAP sums with raw margins.	Log-odds units and numerical additivity; no percentage-point explanation claim.
Retrieval fallback	Missing optional encoder and optional embedding failure; local encoder fixture.	Explicit lexical fallback; an available local encoder yields real embeddings.
Frozen evaluation	Recompute saved test metrics; alter the dataset fingerprint.	Reuse held-out IDs and reject content different from the saved training/evaluation dataset.

Table 6. ML regression procedures represented in ml/tests/test_models.py. Test names and assertions define coverage, not a new representative benchmark.

9.1 Explanation and artifact checks

Text attribution can be checked by comparing the intercept plus all coefficient-times-TF-IDF contributions with decision_function. URL attribution verifies the base-plus-SHAP sum against the raw XGBoost margin. Only a subset of contributions is displayed, so the API returns full-sum information separately. Artifact version, library versions, label order, threshold values and dataset fingerprint support repeatable interpretation.

9.2 What remains to be tested statistically

A stronger ML experiment needs licensed or consented real incident labels, chronological and organization/campaign-disjoint external holdouts, per-language and subgroup outcomes, representative prevalence, and robustness tests for shortened links, obfuscation and OCR corruption. Calibration must use independent development data; threshold policies must be fixed before a new final test. Retrieval and OCR require their own labelled relevance and transcription datasets.

10. API, Security, Media, and Data-Integrity Tests

Existing project QA records report 52 combined backend/ML tests and a separate 31-check live HTTP sequence [10]. These are recorded runs with different scopes, not additive independent trials. The backend suite includes parametrized claim-validation cases; counting function names is not the same as counting collected tests. Most backend integration tests use local fixtures and an isolated test database.

Coverage area	Procedure and asserted behavior
Assessment contract	Thai/English inputs, invalid bodies, multiple identifiers, missing artifacts, raw lexical URL cues and LLM timeout/instruction fixtures. Verify truthful missing data and absence of submitted-URL fetching.
Ownership and sessions	Save history opt-in; access from another user; logout/revocation; profile/export/delete fixtures. Verify owner/admin restrictions and UTC timestamp serialization.
Evidence and moderation	Pending versus reviewed reports, audited reasons, owned attachments and source-status checks. Pending reports do not automatically become admitted threats or training labels.
OCR / QR / payment parsing	Valid and invalid images, unreadable content, QR payloads, CRC validity and receiver preview. Verify an explicit error/confirmation boundary rather than automatic payment.
Batch and exports	Owned job progress and invalid rows; masked exports; raw CSV formula neutralization. Verify per-row reporting and ownership, not distributed execution guarantees.
OIDC and account linking	Generated signed Google tokens; invalid claims/signatures; state/PKCE/nonce and one-use handoff; timeouts, denial, identity reuse and explicit linking. Provider requests are mocked.
LINE webhook / notification	Signature verification; follow/unfollow consent; replay order; masked payloads; idempotency and timeout fixtures. These tests do not establish actual LINE delivery.
Request bounds / caller feed	Chunked-body and rate-limit checks; exclusion of sample, pending, unconfirmed and expired caller entries. The connected feed had no verified real caller records.

Table 7. Backend test methods represented in test_api.py and test_oauth.py. Mocked external integration is explicitly distinguished from real-account activation.

Security measures include salted PBKDF2-SHA256 passwords, opaque session tokens stored as digests, one-use OAuth state/handoff consumption, and owner checks. Evidence images are re-encoded to remove EXIF/GPS; regex masking and CSV escaping reduce disclosure and spreadsheet-injection risks. These mechanisms and regression tests do not amount to an independent penetration audit, universal anonymization, or end-to-end encryption. Web session persistence uses localStorage; native persistence uses SecureStore.

11. Client, PWA, Native, and Runtime Verification

Client QA progressed through recorded revisions: 26 checks for result/input integrity and a later 36-check flow/lifecycle run. These historical counts overlap and should not be summed. Tests exercise the shared interface and mocks; browser and iOS observations supply separate evidence of actual rendering and API communication.

Layer	Test method and recorded outcome / boundary
Shared client	Input/type edits clear prior results; request-generation checks reject stale responses; logout invalidates pending saves/exports/jobs. Save/share/report use the result snapshot, not the subsequently edited draft.
Browser / PWA	Observe login, guest analysis, result rendering and responsive controls. Worker tests enforce a public-asset allowlist and bypass API, authorization, non-GET, OAuth/query and foreign-origin requests. Offline shell is not offline inference.
iOS Simulator	Recorded signed host plus Call Directory extension builds and UI smoke flows on iPhone 17/iOS 26.5 and iPhone 18 Pro/iOS 27. Login, guest API connection, result display, new analysis and foreground return were observed.
iOS lifecycle recovery	A scene adapter addresses an iOS 27 launch failure. Native result surfaces avoid the web entrance-animation path. This is compatibility/interaction evidence, not physical incoming-call evaluation.
Installer / shared runner	Pinned dependency/model readiness; missing prerequisites; API health gates before Metro; explicit URL/port conflicts; server reuse; isolated cancellation and unrelated-process preservation.
Stop / viewer / configuration	PID/path/cwd ownership checks; bounded shutdown; exit-versus-inspection races; DeviceHub/legacy Simulator activation mocks; scene-plugin idempotence and original-template regeneration.
Build/static configuration	TypeScript and web export checks; Vercel routing/environment and CI configuration checks. A checked configuration is not proof of a cloud deployment or a successful hosted CI run.

Table 8. Verification methods and their limits. Physical-device calls, Android compilation, real Google/LINE login and notification delivery, PostgreSQL/container runtime integration, and production deployment remain unverified in the recorded scope.

The final recorded launcher/viewer/scene/stop regression run contained 24, 9, 12 and 15 checks (60 total). Installer and shared-runner suites had separately recorded 6 and 16 checks. Isolated process fixtures test cancellation and shutdown safely; actual npm stop/restart and Simulator UI checks were also recorded. These counts measure tested cases, not statistical accuracy or an exhaustive proof that the software contains no defects.

The prototype uses local background tasks, per-process rate limiting and local model loading. No sustained load test, failure-recovery benchmark, percentile latency measurement, concurrent distributed-worker evaluation, or physical-device power/battery study is reported.

12. Discussion, Limitations, and Future Validation

12.1 Trust and practical interpretation

The interface separates uncalibrated screening scores, explicit history policies, model-margin explanations and provenance relationships. This makes it possible to inspect why an input triggered caution without implying that a graph edge, an unusual URL or a similar message independently proves fraud. Unknown identifiers abstain rather than receive fabricated safe outcomes. Explanation wording should remain restrained even when demonstration metrics are high.

Unresolved question	Required next evaluation
Real-world generalization	Independent verified incident corpus; chronological/campaign-disjoint holdouts; representative benign traffic and prevalence.
Calibration / operating costs	Independent calibration and reliability analysis; false-positive/false-negative costs; thresholds fixed before a final external test.
Language and accessibility	Thai/English subgroup metrics, dialect/obfuscation coverage, readability and accessibility checks, and consented user studies.
Input/retrieval quality	OCR transcription error rates; QR decode success under image degradation; retrieval relevance labels; controlled modality/feature ablations.
Service/phone reliability	Load and fault-injection testing; real provider-account flows; signed physical-device caller tests; database and deployment recovery checks.

Table 9. Evaluation requirements that remain prospective, not completed experiments.

Synthetic generation may embed easy lexical or URL-domain patterns; group separation does not remove all template dependence. Small test support and class balance limit generalization. The graph has no independently trained fraud model, and no accuracy outcome is measured for combined policy decisions. A causal, calibrated, or production-grade interpretation would exceed the available evidence.

13. Conclusion

ScamGraph AI integrates Thai-English text/URL screening with provenance-aware context, reviewed image/QR workflows and shared web/mobile interfaces. The stored synthetic test set yields text accuracy 94.44% (F1 0.9425) and URL accuracy 96.67% (F1 0.9663), with explicit thresholds and reproducible confusion matrices. These component results establish a working experimental pipeline, not reliable detection rates on real scams. A layered verification record supports selected engineering behaviors while clearly separating mocked, observed and unverified capabilities. Real-data validation and deployment studies are the next research requirements.

Author contributions and data/code availability

Kitsanapuch and Thanakorn Morasilp: development; Phutawan Sathusarn: product; Achiraya Sungsriin: UX/UI. This attribution follows the project team credits. No institutional affiliation has been supplied. Code, synthetic data, frozen IDs and metric artifacts are available in the team repository snapshot [10]. No real victim dataset, private credentials or user history is included in this paper. This technical manuscript is not an accepted or peer-reviewed publication.

References

[1] G. Salton and C. Buckley, "Term-weighting approaches in automatic text retrieval," *Information Processing & Management*, 24(5), 513-523, 1988. DOI: 10.1016/0306-4573(88)90021-0.

[2] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, 12, 2825-2830, 2011. jmlr.org/papers/v12/pedregosa11a.html.

[3] T. Chen and C. Guestrin, "XGBoost: A Scalable Tree Boosting System," *Proceedings of KDD*, 785-794, 2016. DOI: 10.1145/2939672.2939785.

[4] F. T. Liu, K. M. Ting, and Z.-H. Zhou, "Isolation Forest," *IEEE International Conference on Data Mining*, 413-422, 2008. DOI: 10.1109/ICDM.2008.17.

[5] S. M. Lundberg and S.-I. Lee, "A Unified Approach to Interpreting Model Predictions," *Advances in Neural Information Processing Systems*, 30, 2017. arXiv:1705.07874.

[6] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks," *EMNLP-IJCNLP*, 3982-3992, 2019. DOI: 10.18653/v1/D19-1410.

[7] N. Reimers and I. Gurevych, "Making Monolingual Sentence Embeddings Multilingual using Knowledge Distillation," *EMNLP*, 4512-4525, 2020. DOI: 10.18653/v1/2020.emnlp-main.365.

[8] Sentence Transformers, "paraphrase-multilingual-MiniLM-L12-v2," official model card, n.d. Hugging Face model card. Verified 9 October 2026. Project-pinned revision: e8f8c211226b894fcb81acc59f3b34ba3efd5f42.

[9] S. Kapoor and A. Narayanan, "Leakage and the reproducibility crisis in machine-learning-based science," *Patterns*, 4(9), 100804, 2023. DOI: 10.1016/j.patter.2023.100804.

[10] ScamGraph AI Project Team, *ScamGraph AI source and verification records*, 2026. Hard2Coding/UHackathon, snapshot 3abce60778fb. Sources: ml/artifacts/default/metadata.json and evaluation.json; ml/pipeline.py and inference.py; backend/app; regression suites; QA_RESULTS.md and FUNCTION_STATUS.md. Repository inspection: 9 October 2026.

Evidence provenance note

Quantitative values come from saved JSON artifacts and deterministic read-only dataset audits. Engineering outcomes are attributed to the project QA record, including its stated gaps. Scholarly references substantiate the component methods; they do not validate this prototype's fraud-detection claims. Documentation and command reproduction are provided in Appendix A.

Appendix A. Reproduction and Complete Test Inventory

Use an isolated development checkout and database. The saved training environment records Python 3.12.14, scikit-learn 1.6.1 and XGBoost 3.0.0. The application snapshot pins Expo 55.0.31 and React Native 0.83.10. Local setup requires Node.js 22 or newer and Python 3.12; native iOS additionally requires macOS/Xcode/CocoaPods. Lockfiles and the embedding revision, rather than current package defaults, should govern reproduction.

Purpose	Existing command / source
Prepare dependencies	npm install (repository root)
Backend + ML regression	source scripts/env.sh; .venv/bin/python -m pytest backend/tests ml/tests -q
Dataset validation	source scripts/env.sh; .venv/bin/python -m ml validate ml/datasets/sample.jsonl
Frozen metric recomputation	.venv/bin/python -m ml evaluate ml/datasets/sample.jsonl --artifacts ml/artifacts/default
Client/type checks	npm run typecheck; npm run test:flows
PWA / static web	npm run test:webapp; npm run test:vercel; npm run build:web
Local process / installer	npm run test:dev; npm run test:install
Native / shutdown regressions	npm run test:ios; npm run test:stop
Observed smoke workflow	npm run dev (web); npm run ios (Simulator); npm run stop (verified project servers)
Combined project checks	npm run check (includes evaluation and OpenAPI/web export steps)

Table A1. Reproduction commands for the versioned repository.

Evaluation and the combined check script write output artifacts; run them in a disposable checkout to preserve the published snapshot. Tests may create isolated databases and temporary artifacts. The example commands document the existing protocol and were not executed as new paper experiments. Do not report regenerated metrics as identical unless the dataset fingerprint, frozen IDs, software versions and thresholds match.

Minimum audit before reporting a new result

1. Validate labels, source permissions and sample flags; compute the canonical fingerprint.
2. Establish connected campaign/domain/text groups; fix training, validation and test IDs.
3. Fit transformations and models only on training rows; retain training-only references.
4. Select thresholds on validation; record class counts, label order and software versions.
5. Evaluate the untouched test set once for the declared protocol; report counts and all metrics.
6. Verify explanation units, missing-data states, ownership and stale-response handling.
7. Separate mocks, real browser/Simulator observations and untested field capabilities.
8. Preserve a versioned result and repeat on an independent real-data/temporal holdout before a production-accuracy claim.

The inventory above includes available ML, API/OAuth, client, PWA, deployment-configuration and local/native-process checks. "All tests" describes the documented test layers, not exhaustive exploration of every input, attack, device, network condition or deployment configuration.